

Clean Code Notes: Objects and Data Structures

Extended Version – Deep Explanation + Bad & Good Code

1. Data Abstraction

Abstraction is not about getters/setters. It is about hiding implementation and exposing intention.

A good abstraction lets users interact with the object without knowing internal details.

This allows flexibility and easier future changes.

Bad Example

```
public class Point {  
    public double x;  
    public double y;  
}
```

Good Example

```
public interface Point {  
    double getX();  
    double getY();  
    void setCartesian(double x, double y);  
    double getR();  
    double getTheta();  
    void setPolar(double r, double theta);  
}
```

2. Abstract vs Concrete Interface

Concrete interfaces expose internal representation.

Abstract interfaces expose meaning and hide calculation details.

Bad Example

```
public interface Vehicle {  
    double getFuelTankCapacityInGallons();  
    double getGallonsOfGasoline();  
}
```

Good Example

```
public interface Vehicle {  
    double getPercentFuelRemaining();  
}
```

3. Data vs Object

Data structures expose data and have no behavior.

Objects hide data and expose behavior.

Choosing between them depends on system needs.

Data Structure

```
class Square {  
    public double side;  
}
```

Object

```
class Square {  
    private double side;  
    public double area() {  
        return side * side;  
    }  
}
```

4. Procedural vs OOP

Procedural code makes it easy to add new functions but hard to add new types.

OOP makes it easy to add new types but harder to add new functions.

Bad (Procedural)

```
public double area(Object shape) {  
    if (shape instanceof Square) {  
        return ((Square) shape).side * ((Square) shape).side;  
    }  
}
```

Good (OOP)

```
interface Shape {  
    double area();  
}  
  
class Square implements Shape {  
    private double side;  
    public double area() {  
        return side * side;  
    }  
}
```

5. Law of Demeter

A method should only talk to its immediate collaborators.

Avoid deep chaining of method calls.

Prefer telling objects what to do instead of asking for data.

Bad

```
ctxt.getOptions().getScratchDir().getAbsolutePath();
```

Best

```
BufferedOutputStream bos = ctxt.createScratchFileStream(fileName);
```

6. Hybrid Structures

Hybrid structures mix data exposure and behavior.

They are hard to maintain and should be avoided.

```
class User {  
    public String name;  
    public void calculateSomething() {}  
}
```

7. DTO

DTOs are simple data carriers.

They should not contain business logic.

```
class UserDTO {  
    public String name;  
    public int age;  
}
```

8. Active Record

Active Record combines data with database methods.

Business logic should not be placed here.

```
class User {  
    public String name;  
    public void save() {}  
    public static User find() {}  
}
```

9. Key Takeaways

- 1 Objects hide data and expose behavior
- 2 Data structures expose data
- 3 Use abstraction over getters/setters
- 4 Follow Law of Demeter
- 5 Avoid hybrids
- 6 DTO for data transfer only
- 7 Choose design based on change patterns